



Universidade de Brasília
Instituto de Exatas
Departamento de Ciência da Computação

Relatório do Laboratório 3

Processador Multiciclo

CIC0099 - Organização e Arquitetura de Computadores

Turma 02 - Prof. Dr. Ricardo Pezzuol Jacobi

Grupo B4

Henrique Morcelles Salum	232003008
Athos Calixto Muniz	211068261
Gabriela Fernanda Rodrigues Costa	180120859
Cauê Araújo Euzebio	211028195
Lucas Silva Nóbrega	180035096

Sumário

1	Introdução	2
2	Métodos e Análise	2
2.1	Experimento	2
2.1.1	Tarefa I	3
2.1.2	Tarefa II	3
2.1.3	Tarefa III	5
2.1.4	Tarefa IV	9
3	Conclusão	11

1 Introdução

O presente experimento tem como objetivo introduzir os alunos ao processo de projeto e implementação de sistemas digitais utilizando VHDL como Linguagem de Descrição de Hardware (HDL). A atividade busca promover o treinamento prático em modelagem, análise e síntese de circuitos digitais, utilizando o ambiente de desenvolvimento **Intel Quartus Prime v24.1** como ferramenta principal.

Como aplicação prática, propõe-se a implementação de um processador Multiciclo compatível com a ISA RV32I reduzida, contendo as instruções: `add`, `sub`, `and`, `or`, `slt`, `lw`, `sw`, `beq`, `jal`, `jalr`, `addi` e `lui`. Essa abordagem permite compreender o funcionamento interno de um processador Multiciclo, ou seja, a unidade de controle, o caminho de dados e o fluxo de instruções aplicados na divisão de instruções em etapas.

A implementação será validada por meio de simulação funcional e temporal, com análise de *timing* e frequência máxima de operação, permitindo compreender, de forma integrada, o funcionamento de uma arquitetura RISC-V.

2 Métodos e Análise

2.1 Experimento

Enunciado

Implemente o processador Multiciclo com ISA Reduzida com as instruções: `add`, `sub`, `and`, `or`, `slt`, `lw`, `sw`, `beq`, `jal`, e ainda as instruções `jalr`, `addi` e `lui`.

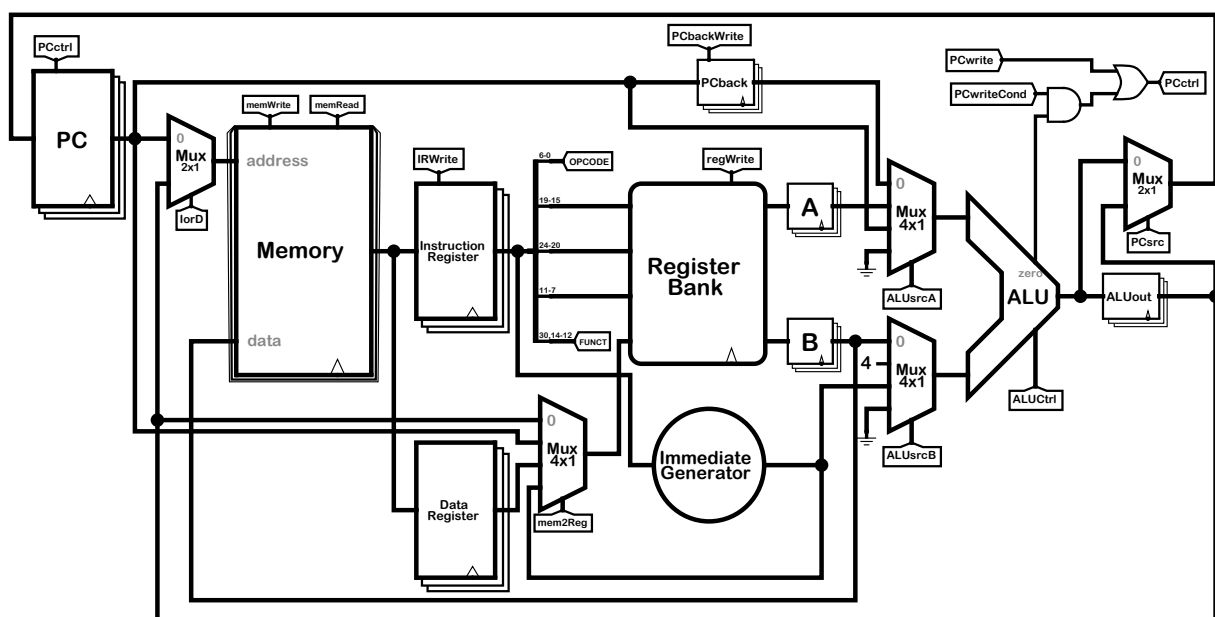


Figura 1: Esquemático do Multiciclo com ISA reduzida

Para realizar o solicitado, é-nos fornecido o esquemático de uma implementação do Multiciclo com ISA reduzida. Nesse esquemático, porém, foi necessário realizar alterações no caminho de dados, que serão devidamente justificadas adiante. Em que os túneis representam entradas e saídas do módulo de controle do processador, que foi abstraído na figura acima e será devidamente explicado na seção adequada.

2.1.1 Tarefa I

Enunciado

Na Arquitetura Von Neumann, dados e programas na mesma memória, implica que a memória de código deve ser gravável do mesmo modo que a memória de dados. Logo basta colocar o controle igual das duas memórias, apenas selecionando, pelo endereço, qual dos dois blocos deve ser utilizado. Explique como isso foi feito no seu processador.

Como utilizamos a mesma memória do Uniciclo, dividida em `ramI`, de instruções e `ramD`, de dados, de fato, ainda há separação entre dados e memória. A estratégia para unificar a memória foi criar uma entidade `unifiedMemory`, que cria uma abstração de Arquitetura Von Neumann para o resto do sistema - o processador só vê um bloco de memória, que tem como entradas um endereço, um dado, o *clock* e duas entradas de controle: `memWrite` e `memRead`.

Dentro dessa unidade, então, verifica-se se o objetivo é acessar instruções ou dados por meio do bit 28 do endereço. Esse mesmo bit é utilizado para, por meio de operações `and` envolvendo ele e os sinais de controle, decidir qual memória de fato é acessada e, via atribuição condicional, decidir se a saída da memória unificada será a da memória de dados ou de instruções.

2.1.2 Tarefa II

Enunciado

O bloco de memória IP utilizado pelo Quartus necessita 2 ciclos de clock para acesso. Explique que alterações podemos fazer no Diagrama de Estados de modo a otimizar esse acesso.

Para o acesso ao bloco de memória, são necessários dois ciclos de clock. Sabendo que o acesso se refere tanto à leitura quanto à escrita, é possível realizar algumas mudanças no diagrama de estados do Bloco de Controle de forma a otimizar esse acesso.

Anteriormente, no projeto do Processador Uniciclo, a resolução para o dado problema foi dividir o clock em dois: um clock exclusivamente para a memória e um outro clock para o resto da CPU. Seguindo essa ideia, o clock da memória, ao ser configurado para

ter uma frequência duas vezes maior que a frequência do processador, foi capaz de suprir a sua necessidade por dois ciclos sem interferir no funcionamento da CPU.

Agora, no projeto do Processador Multiciclo, não é mais necessário dividir o clock em dois. Isso porque iremos otimizar o acesso à memória a partir do Bloco de Controle, especificamente, a partir do seu diagrama de estados. Para realizarmos essa otimização, primeiro, é necessário identificar quais são os estados que realizam o acesso à memória. Como esse acesso é caracterizado tanto pela escrita quanto pela leitura, basta observar quais estados ligam qualquer um dos sinais `memRead` ou `memWrite`. Assim, foram identificados os seguintes estados:

- **Fetch:** É o estado inicial do diagrama, ou seja, o primeiro a ser executado. Esse estado liga o sinal `readMem` para que a próxima instrução seja lida pelo bloco de memória;
- **MemRead:** É o estado caracterizado por ser a quarta etapa de uma instrução “load”. Esse estado liga o sinal `LeMem` para que o bloco de memória leia o dado escolhido para que esse dado seja escrito em um registrador no próximo estado;
- **MemWrite:** É o estado caracterizado por ser a quarta e última etapa de uma instrução “store”. Esse estado liga o sinal `memWrite` para que o bloco de memória sobrescreva o dado escolhido.

Identificados os estados, podemos implementar a seguinte otimização: criar novos estados iguais a esses (ou seja, com as mesmas saídas), posicionando estes imediatamente antes daqueles no diagrama de estados. Assim, segue um exemplo: o diagrama de estados terá, como o estado inicial, a “cópia” do Fetch (que ativa os mesmos sinais que o Fetch “original”, incluindo o sinal `memRead`) e, quando chegar a batida do clock, o novo estado será o próprio Fetch “original”.

Dessa forma, estaremos garantindo que os estados que acessam a memória utilizem dois ciclos, sem, ao contrário do uniciclo, que seja necessário retardar o tempo de clock do CPU para que ele seja correspondente ao de dois clocks de memória. Destarte, poder-se-á utilizar a maior frequência de clock possível.

Apesar dessa ser uma otimização relevante, ela não foi realizada por este grupo, pois, quando percebemos essa possibilidade, o projeto do controle já estava demasiado avançado, de forma que não era viável reiniciá-lo considerando os novos fatos.

2.1.3 Tarefa III

Enunciado

Implemente o Bloco Controlador e desenhe a máquina de estados do controle.

Como o enunciado adianta, o bloco de controle do Multiciclo é implementado via máquina de estado. Particularmente, utilizamos uma máquina de estados finita do tipo Moore (isto é, em que a saída depende exclusivamente do estado atual, não das entradas). Para projetar uma máquina de estados, primeiro precisamos do seu diagrama de estados; o diagrama de estados completo do bloco de controle do Multiciclo com ISA reduzida, considerando as doze instruções solicitadas, é mostrado na Figura 2.

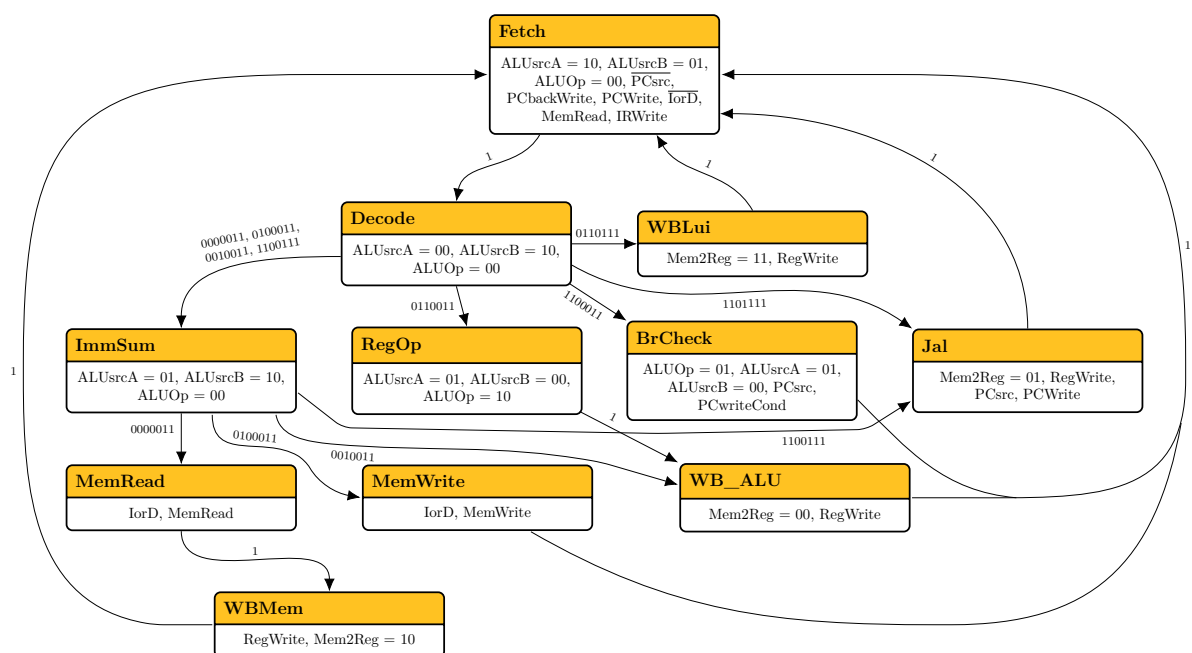


Figura 2: Diagrama de estados do controle

A partir desse diagrama, conseguimos montar as tabelas (1) de saída e (2) de transição de estados, a partir das quais é possível inferir as lógicas de saída e de próximo estado, respectivamente.

Estado Atual	Saídas													
	PCsrc	ALUOp	ALUsrcA	ALUsrcB	PCbackWrite	RegWrite	Mem2Reg	PCwriteCond	PCWrite	IorD	MemWrite	MemRead	IRWrite	
0000	0	00	10	01	1	0	XX	0	1	0	0	1	1	
0001	X	00	00	10	0	0	XX	0	0	X	0	0	0	
0011	X	00	01	10	0	0	XX	0	0	X	0	0	0	
0010	X	10	01	00	0	0	XX	0	0	X	0	0	0	
1000	1	01	01	00	0	0	XX	1	0	X	0	0	0	
0101	1	XX	XX	XX	0	1	01	0	1	X	0	0	0	
0110	X	XX	XX	XX	0	0	XX	0	0	1	0	1	0	
0111	X	XX	XX	XX	0	0	XX	0	0	1	1	0	0	
1001	X	XX	XX	XX	0	1	00	0	0	X	0	0	0	
1011	X	XX	XX	XX	0	1	10	0	0	X	0	0	0	
1101	0	00	10	01	0	1	11	0	0	X	0	0	0	

Tabela 1: Tabela verdade das saídas do controle

Com base nesta tabela, utilizamos Mapas de Karnaugh para minimizar, na forma de soma de produtos, as expressões para as saídas. Os resultados obtidos estão exibidos a seguir, em que Q_i representa o i -ésimo bit do estado atual.

$$PCsrc = \overline{Q_3} Q_2 + Q_3 \overline{Q_2} \overline{Q_1}$$

$$ALUOp_1 = \overline{Q_3} Q_1 \overline{Q_0}, ALUOp_0 = Q_3 \overline{Q_2} \overline{Q_1}$$

$$ALUsrcA_1 = Q_2 \overline{Q_1} Q_0 + \overline{Q_3} \overline{Q_2} \overline{Q_1} \overline{Q_0}, ALUsrcA_0 = \overline{Q_3} Q_1 + Q_3 \overline{Q_2} \overline{Q_1}$$

$$ALUsrcB_1 = \overline{Q_3} Q_0, ALUsrcB_0 = Q_2 \overline{Q_1} Q_0 + \overline{Q_3} \overline{Q_2} \overline{Q_1} \overline{Q_0}$$

$$PCbackWrite = \overline{Q_3} \overline{Q_2} \overline{Q_1} \overline{Q_0}$$

$$RegWrite = Q_3 \overline{Q_2} Q_0 + Q_2 \overline{Q_1} Q_0$$

$$Mem2Reg_1 = \overline{Q_2} Q_1 Q_0 + Q_3 Q_2 \overline{Q_1} Q_0, Mem2Reg_0 = Q_2 \overline{Q_1} Q_0$$

$$PCWriteCond = Q_3 \overline{Q_2} \overline{Q_1} \overline{Q_0}$$

$$PCWrite = \overline{Q_3} \overline{Q_2} \overline{Q_1} \overline{Q_0} + \overline{Q_3} Q_2 \overline{Q_1} Q_0$$

$$IorD = \overline{Q_3} Q_1$$

$$MemWrite = \overline{Q_3} Q_2 Q_1 Q_0$$

$$MemRead = \overline{Q_3} \overline{Q_2} \overline{Q_1} \overline{Q_0} + \overline{Q_3} Q_2 Q_1 \overline{Q_0}$$

$$IRWrite = \overline{Q_3} \overline{Q_2} \overline{Q_1} \overline{Q_0}$$

Tendo essas equações, é fácil implementar a lógica de saída. O esquemático a seguir mostra essa implementação.

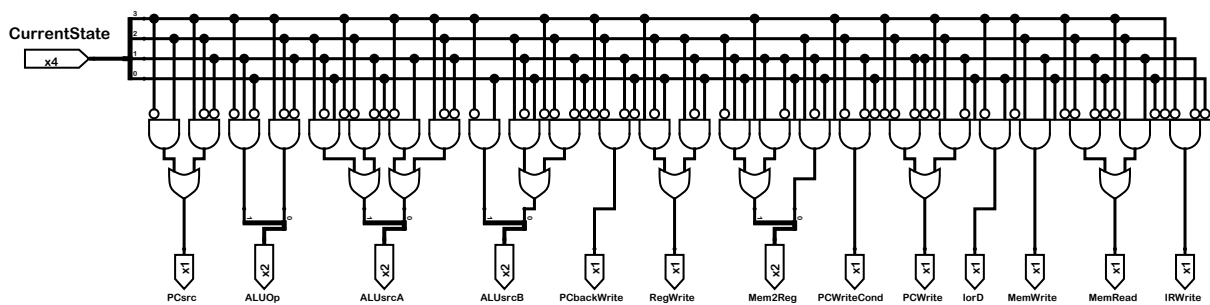


Figura 3: Esquemático da lógica de saída do controle

O circuito acima foi separado, tanto neste relatório (nos esquemáticos) quanto no projeto em VHDL em uma entidade própria, chamada `OutLogic`. Ela será instanciada, assim como a lógica de próximo estado, que será explicada adiante, no bloco de controle como um componente com entrada `currentState` e as saídas representadas acima.

Estado Atual	Próximo Estado							
	0000011	0100011	0010011	1100111	0110011	1100011	1101111	0110111
0000	0001	0001	0001	0001	0001	0001	0001	0001
0001	0011	0011	0011	0011	0010	1000	0101	1101
0011	0110	0111	1001	0101	X	X	X	X
0010	X	X	X	X	1001	X	X	X
1000	X	X	X	X	X	0000	X	X
0101	X	X	X	0000	X	X	0000	X
0110	1011	X	X	X	X	X	X	X
0111	X	0000	X	X	X	X	X	X
1001	X	X	0000	X	0000	X	X	X
1011	X	0000	X	X	X	X	X	X
1101	X	X	X	X	X	X	X	0000

Tabela 2: Tabela de transição de estados do controle

A abordagem para implementar a lógica de próximo estado é um pouco diferente; não minimizaremos equações nem nada do tipo. Na verdade, tendo definido que um registrador de deslocamento será utilizado como elemento de memória, montamos outra tabela a partir da Tabela 2; uma em que, ao invés do próximo estado, representamos, em cada célula, a operação que o registrador precisa fazer para chegar àquele estado.

O Registrador de Deslocamento é capaz de fazer cinco operações: *shift left(L)*, *shift left(R)*, *load(D)*, *hold* e *reset*, em que a variável entre parênteses representa o argumento necessário para aquela operação (L: o bit que entra a esquerda no *shift left*, R: o bit que entra a direita *shift left*, D: o vetor de quatro bits que substitui o estado atual no *load*). O *hold* e o *reset* não foram utilizados nesse projeto.

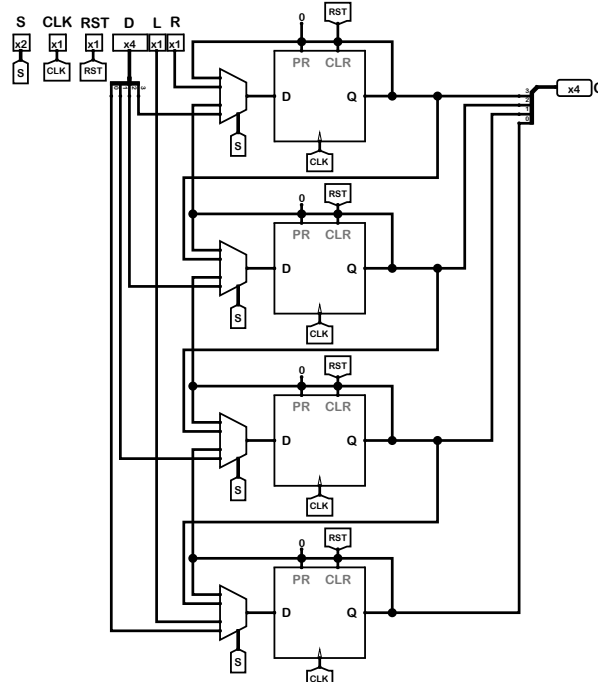


Figura 4: Lógica interna do Registrador de Deslocamento

Estado Atual	Próximo Estado							
	0000011	0100011	0010011	1100111	0110011	1100011	1101111	0110111
0000	SL(1)	SL(1)	SL(1)	SL(1)	SL(1)	SL(1)	SL(1)	SL(1)
0001	SL(1)	SL(1)	SL(1)	SL(1)	SL(0)	SR(1)	LOAD(0101)	LOAD(1101)
0011	SL(0)	SL(1)	SR(1)	LOAD(0101)	X	X	X	X
0010	X	X	X	X	SR(1)	X	X	X
1000	X	X	X	X	X	SL(0)	X	X
0101	X	X	X	LOAD(0000)	X	X	LOAD(0000)	X
0110	SR(1)	X	X	X	X	X	X	X
0111	X	LOAD(0000)	X	X	X	X	X	X
1001	X	X	LOAD(0000)	X	LOAD(0000)	X	X	X
1011	X	LOAD(0000)	X	X	X	X	X	X
1101	X	X	X	X	X	X	X	LOAD(0000)

Tabela 3: Tabela de transição de estados do controle

Note que os estados foram escolhidos *a priori* de forma a minimizar os ‘LOAD’s necessários - claro que não há garantia de que essa minimização foi ótima, ela foi feita por inspeção.

Agora, basta utilizar um multiplexador de tamanho adequado para selecionar as entradas corretas para o registrador de deslocamento em cada caso, isto é, os valores de OP (S, na Figura 4), L, R e D (que dependem, claro, da operação realizada) - o trem de bits que entra nos multiplexadores foi montado nessa ordem. Dessa forma, o circuito a seguir foi montado; chamamos ele de *Next State Logic*, mas, de fato, ele implementa a lógica da entrada do Registrador de Deslocamento, não do próximo estado *per se*.

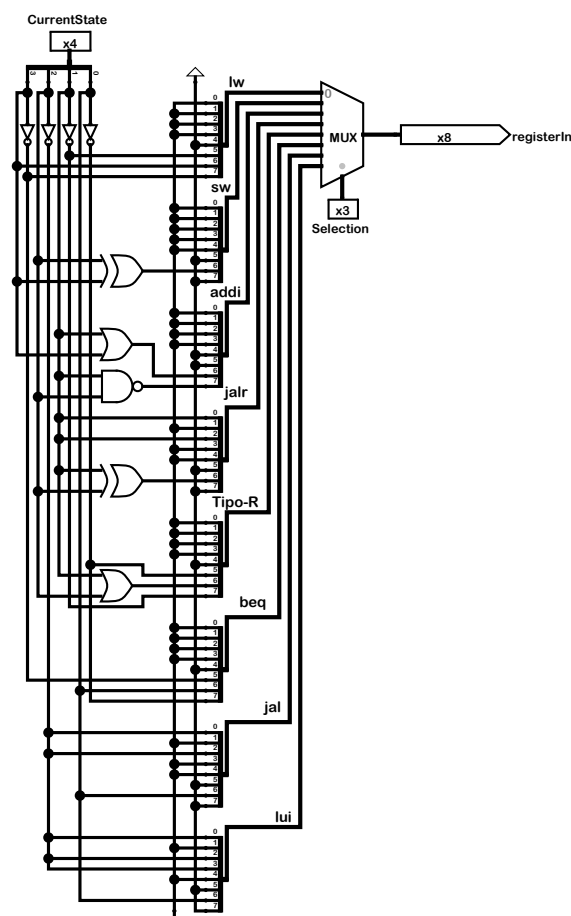


Figura 5: Lógica do próximo estado

Finalmente, o bloco de controle, internamente, ficou como na Figura 6: abstrai-se as lógicas de próximo estado e de saída, que foram explicadas acima, conecta-se às entradas do Registrador de Deslocamento as saídas daquele e à saída, as entradas deste.

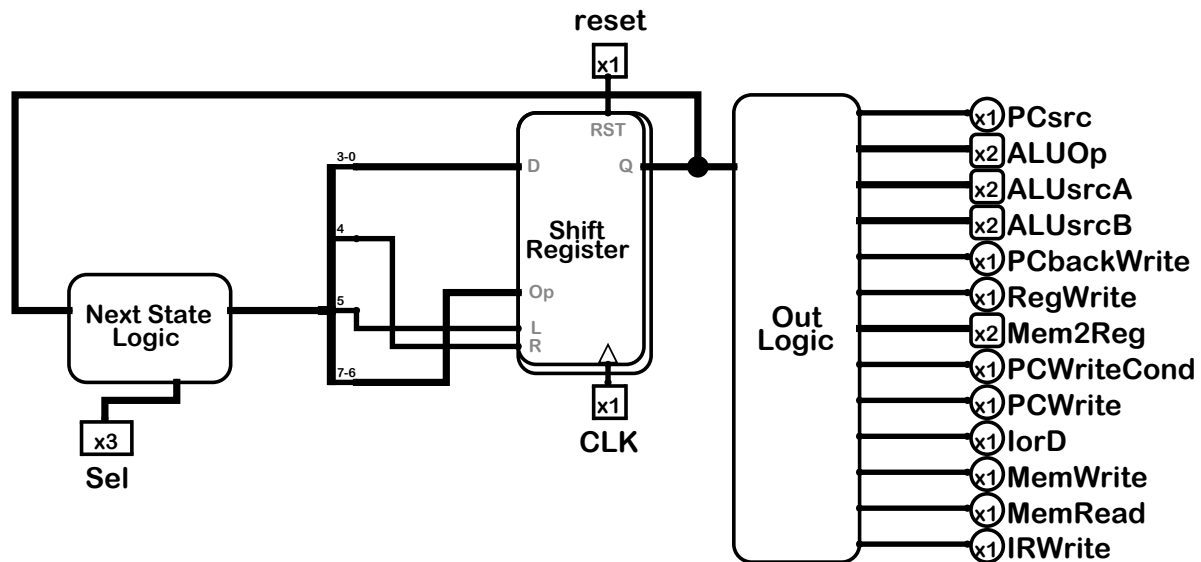


Figura 6: Bloco de Controle do multiciclo

2.1.4 Tarefa IV

Enunciado

Implemente o Processador Multiciclo completo.

- Visualize os blocos funcionais com o netlist RTL view.
- Levante os requisitos físicos e temporais do seu processador.
- Faça a simulação por forma de onda funcional e temporal com o programa `de1.s`, mostrando o funcionamento correto da CPU.
- Qual a máxima frequência de clock utilizável na sua CPU? Verifique experimentalmente mudando a frequência de `CLOCK` e apresentando a simulação temporal por forma de onda.

(a) No que tange a visualização com o netlist RTL view, de maneira semelhante ao que foi apresentado no Laboratório 2, inicialmente se destaca o maior nível hierárquico do projeto, o qual consiste de uma entrada `CLOCK` que, com auxílio de um flip-flop tipo D, permite ser dividido para valores diferentes para a memória e processador. Além das entradas `reset` e `regin`, que descrevem respectivamente a entrada para reiniciar os registradores do sistema e a entrada que descreve os valores utilizados em `t0`.

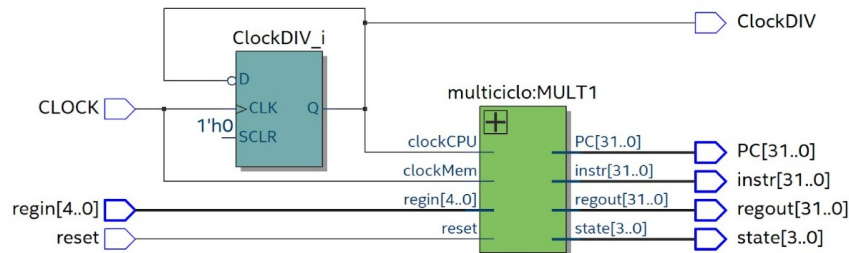


Figura 7: Netlist do TopDE

Além disso, ao visualizar a estrutura interna do processador Multiciclo, é perceptível os diferentes módulos instanciados, sobretudo os bancos de registradores, o bloco controlador, a ULA, os diferentes multiplexadores, entre outros.

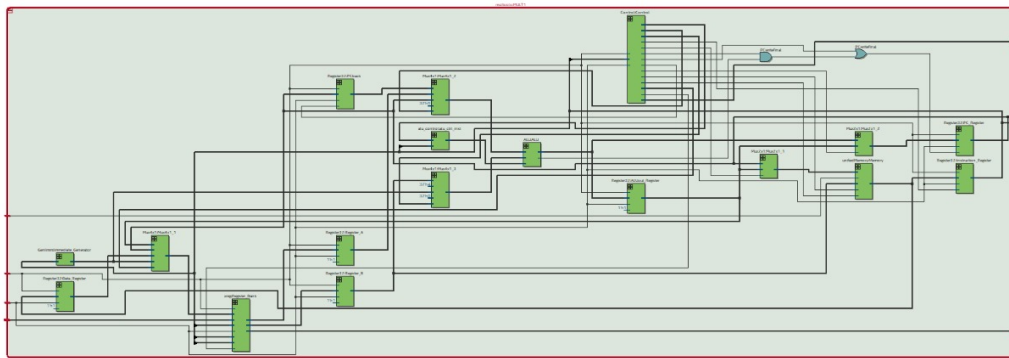


Figura 8: Netlist do Multiciclo

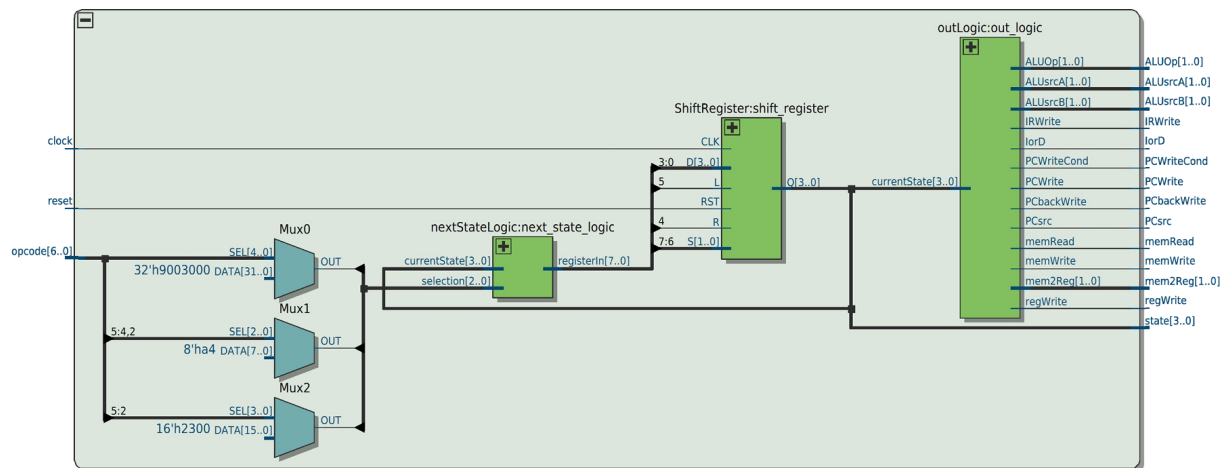


Figura 9: Netlist do bloco de controle

(b) Para os requisitos físicos, de maneira sucinta, foram utilizados 3291 elementos lógicos, 1253 registradores, 108 pinos, além de 65536 bits de memória. Destacando que, como não foi necessário utilizar multiplexadores mais complexos. Já para os requisitos temporais foi avaliado que os slacks de setup e hold são cumpridos, respectivamente possuindo 6,69 ns e 2,715 ns. Já a frequência máxima de operação obtida, utilizando a forma que obtêm essa resposta de maneira mais robusta, foi referente à 302,11 MHz.

(c) As simulações, utilizando o ModelSim, sob a frequência de clock de 50 MHz (20 ns de período):

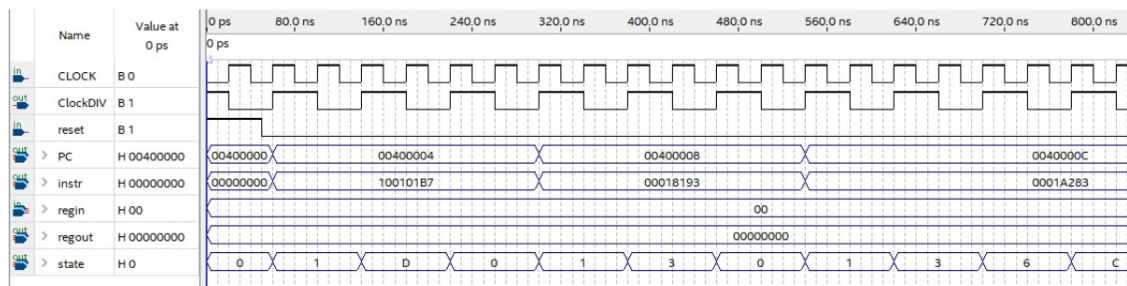


Figura 10: Forma de onda da simulação funcional

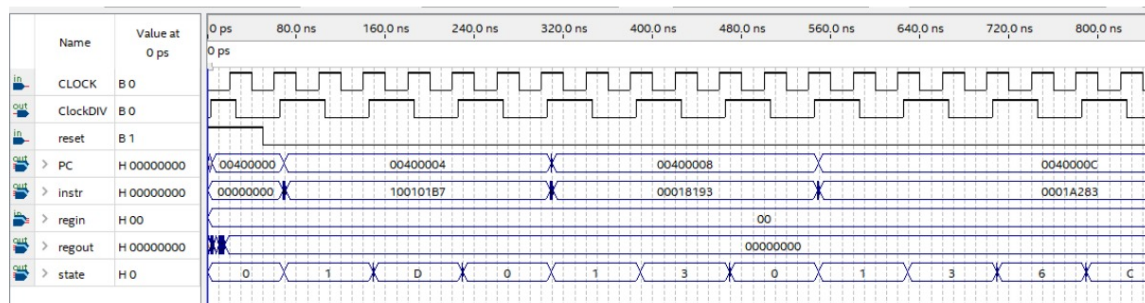


Figura 11: Forma de onda da simulação temporal

(d) Assim como foi obtido anteriormente, a frequência máxima de operação do processador é de 302,11 MHz, verificando-se:

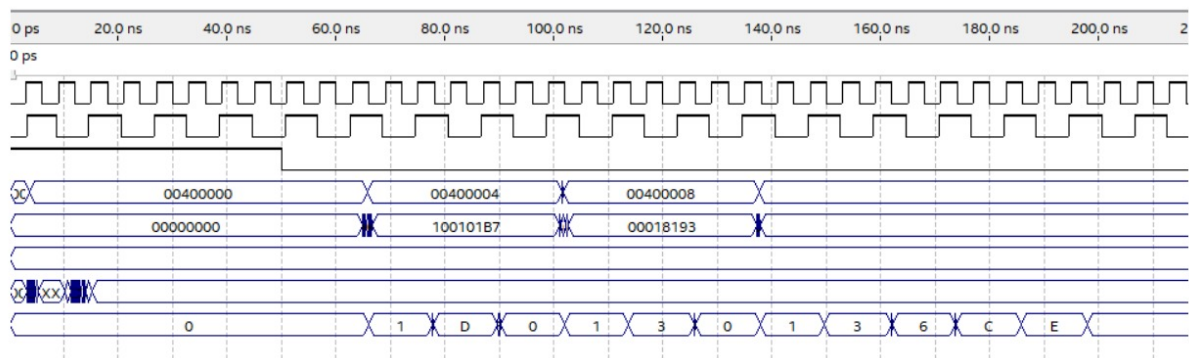


Figura 12: Forma de onda da simulação temporal

3 Conclusão

Todos os códigos desenvolvidos para a realização desse laboratório foram enviados em anexo a esse relatório compactados em um arquivo .zip. Para acessar o vídeo solicitado, [clique aqui](#).